

B28: SQL SuperHero Program : Session 40

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

2 May 2025 at 07:15

NVL Function:

NVL function lets you substitute a value when a null value is encountered.
The syntax for the NVL function in Oracle/PLSQL is:

```
NVL( string1, replace_with )
```

Parameters or Arguments

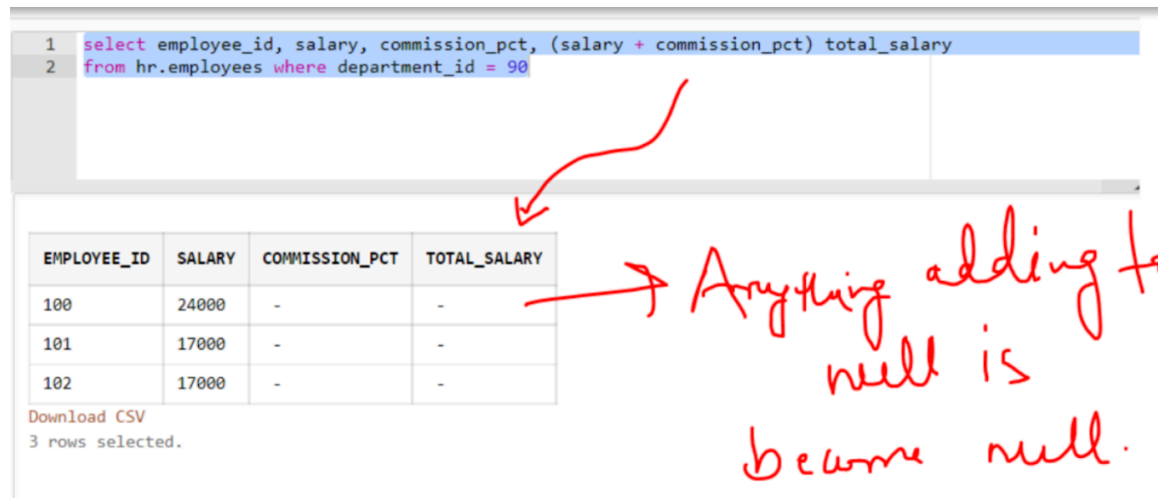
string1

The string to test for a null value.

replace_with

The value returned if string1 is null.

```
select employee_id, salary, commission_pct, (salary + commission_pct) total_salary  
from hr.employees where department_id = 90;
```



EMPLOYEE_ID	SALARY	COMMISSION_PCT	TOTAL_SALARY
100	24000	-	-
101	17000	-	-
102	17000	-	-

Download CSV
3 rows selected.

Anything adding to null is become null.

NVL help to replace null value to anything

```

1 select employee_id, salary, commission_pct, nvl(commission_pct,0)
2 from hr.employees where department_id in (90,80)
3
4

```

EMPLOYEE_ID	SALARY	COMMISSION_PCT	NVL(COMMISSION_PCT,0)
100	24000	-	0
101	17000	-	0
102	17000	-	0
145	14000	.4	.4
146	13500	.3	.3
147	12000	.3	.3
148	11000	.3	.3
149	10500	.2	.2
150	10000	.3	.3

See the above example, null value for first 3 rows are replaced with 0.

=====

```

1 select employee_id, manager_id from hr.employees where department_id = 90
2
3

```

EMPLOYEE_ID	MANAGER_ID
100	-
101	100
102	100

[Download CSV](#)

3 rows selected.

requirement is to null manager id change to 9999.

SQL Worksheet

```
1 select employee_id, manager_id, nvl(manager_id, 9999) from hr.employees where department_id = 90
```

EMPLOYEE_ID	MANAGER_ID	NVL(MANAGER_ID,9999)
100	-	9999
101	100	100
102	100	100

Download CSV

3 rows selected.

NVL replace with same data type. If want to replace with different datatype, it gives error.

```
select employee_id, nvl(manager_id,'ABC') from hr.employees where department_id = 90;
```

In the above query, I want to replace the null value in manager_id column with 'ABC'. manager_id is number and 'ABC' is varchar. NVL will execute everytime to check the 2nd parameter.

```
1 select employee_id, nvl(manager_id,'ABC') from hr.employees
2 where department_id = 90
```

error. bcoz different
datatype

ORA-01722: invalid number

```
create table test (mydate date, yourdate varchar2(20));
```

```
insert into test values ('20-SEP-2023', null);
```

```
insert into test values (null, '18-OCT-2022');
```

I have created a table and inserted record. If you see one column in date column and other is varchar2. Remember one line NVL may be not apply (untill did not get a null value to replace, but evaluate for each row).

```
select * from test
```

MYDATE	YOURDATE
20-SEP-23	-
-	18-OCT-2022

2 rows selected.

Handwritten notes:
 - Orange arrow pointing to MYDATE: **Date datatype**
 - Green arrow pointing to YOURDATE: **Varchar datatype**

```
select mydate, yourdate, nvl(yourdate, '30-FEB-2022') custom_date from test
```

MYDATE	YOURDATE	CUSTOM_DATE
20-SEP-23	-	30-FEB-2022
-	18-OCT-2022	18-OCT-2022

2 rows selected.

Handwritten notes:
 - Green arrow pointing to '30-FEB-2022': **Varchar**
 - Green arrow pointing to the result: **Consider as string**
 - Green checkmark and text: **work fine**
 - Purple arrow pointing to '30-FEB-2022': **Date datatype**
 - Purple arrow pointing to the result: **not a valid date & gives error**

```
select mydate, yourdate, nvl(mydate, '30-FEB-2022') custom_date from test
```

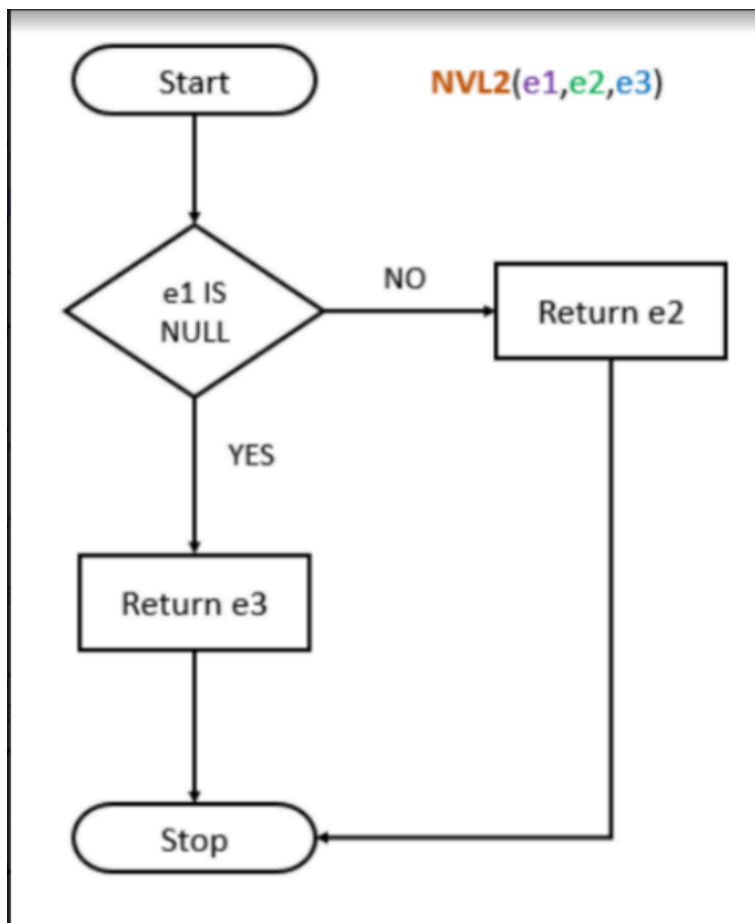
ORA-01830: date format picture ends before converting entire input string

More Details: <https://docs.oracle.com/error-help/db/ora-01830>

NVL2 Function

The Oracle NVL2() function is an extension of the NVL() function. The Oracle NVL2() function accepts three arguments. If the first argument is not null, then it returns the second argument. In case the second argument is null, then it returns the third argument.

The following illustrates the syntax of the Oracle NVL2() function:



`NVL2( string1, value_if_not_null, value_if_null );`

```
select NVL2(null,1,2) from dual
```

NVL2(NULL,1,2)
2

--> In this above example it will return 2, because the first parameter is **null**. NVL2 will check if first parameter is not null then 2nd else 3rd. remember, `NVL2( string1, value_if_not_null, value_if_null );`

```
select NVL2(3,1,2) from dual
```

NVL2(3,1,2)
1

-> In this above example it will return 3, because the first parameter is **not null**. NVL2 will check if first parameter is not null then 2nd else 3rd. remember, **NVL2(string1, value_if_not_null, value_if_null);**

COALESCE Function

The Oracle/PLSQL COALESCE function returns the first non-null expression in the list. If all expressions evaluate to null, then the COALESCE function will return null.

It can take any number of arguments , and see which value is first not null , it returns that one.

```
1 select coalesce(null,2,3,null,3) from dual;
```

COALESCE(NULL,2,3,NULL,3)
2

[Download CSV](#)

In this above example , the first not null value is 2, so it returns 2.

```
CREATE TABLE emergency_contacts (
  contact_id NUMBER GENERATED BY DEFAULT AS IDENTITY,
  employee_id NUMBER NOT NULL,
  first_name VARCHAR2(100) NOT NULL,
  last_name VARCHAR2(100) NOT NULL,
  relationship VARCHAR2(100),
  home_phone VARCHAR2(25),
  work_phone VARCHAR2(25),
  cell_phone VARCHAR2(25)
);
```

```
INSERT INTO emergency_contacts ( employee_id, first_name, last_name, relationship, home_phone, work_phone, cell_phone )
VALUES ( 1,
        'Mary',
        'Bailey',
```

```
'Wife',
NULL,
'515.123.4568',
'515.123.4569');
```

```
INSERT INTO emergency_contacts ( employee_id, first_name, last_name, relationship, home_phone, work_phone, cell_phone )
VALUES ( 2,
'John',
'Rivera',
'Husband',
NULL,
NULL,
'515.123.3563' );
```

```
INSERT INTO emergency_contacts ( employee_id, first_name, last_name, relationship, home_phone, work_phone, cell_phone )
VALUES ( 3,
'Joan',
'Cooper',
'Mother',
NULL,
NULL,
NULL );
```

```
1 select * from emergency_contacts;
2
```

CONTACT_ID	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	RELATIONSHIP	HOME_PHONE	WORK_PHONE	CELL_PHONE
1	1	Mary	Bailey	Wife	-	515.123.4568	515.123.4569
2	2	John	Rivera	Husband	-	-	515.123.3563
3	3	Joan	Cooper	Mother	-	-	-

Download CSV

3 rows selected.

I want to choose one phone number here, which is the first not null value.

```
1 SELECT c.*, COALESCE(home_phone, work_phone, cell_phone) phone
2 FROM emergency_contacts c
```

which ever first not null

CONTACT_ID	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	RELATIONSHIP	HOME_PHONE	WORK_PHONE	CELL_PHONE	PHONE
1	1	Mary	Bailey	Wife	-	515.123.4568	515.123.4569	515.123.4568
2	2	John	Rivera	Husband	-	-	515.123.3563	515.123.3563
3	3	Joan	Cooper	Mother	-	-	-	-

Download CSV

3 rows selected.

Replace VS Translate

1. REPLACE

- **Purpose:** Replaces a **full substring** (sequence of characters) with another substring in a string.

REPLACE(source_string, search_string, replacement_string)

SELECT REPLACE('Finance Report', 'Report', 'Summary') AS result FROM dual;

The screenshot shows a SQL query in the editor: `SELECT REPLACE('Finance Report', 'Report', 'Summary') AS result FROM dual;`. The result grid displays 'Finance Summary'. A red arrow points from the handwritten text 'Report → replaced by Summary' to the word 'Report' in the query string. Another red arrow points from the same text to the word 'Summary' in the result.

RESULT
Finance Summary

Download CSV

2. TRANSLATE

- **Purpose:** Replaces **individual characters** in a string **character by character**, based on a mapping.

TRANSLATE(source_string, from_string, to_string)

The screenshot shows a SQL query: `SELECT TRANSLATE('Finance123', '123', 'ABC') AS result FROM dual;`. The result grid shows 'FinanceABC'. Handwritten red text next to the query explains the mapping: '1 — A', '2 — B', and '3 — C'.

RESULT
FinanceABC

The screenshot shows a SQL query: `SELECT TRANSLATE('2Finance123', '123', 'ABC') AS result FROM dual;`. The result grid shows 'BFinanceABC'. Handwritten green text next to the query explains the mapping: 'every 1 — A', '2 — B', and '3 — C'.

RESULT
BFinanceABC

Download CSV